

IKB42603 Cloud Computing Security Essentials

Practical Assessment — Question Paper Part 1

Hands-on technical assessment · Kubernetes · Secrets · Multi-Tenancy

Student name	SYED ABDULLAH MUBARAK BIN SYED OMAR SHUKRI
Student ID	52215225160
Question(s) attempted	1
Date and session	11/9/2026

Instructions to candidates

- This is a practical assessment. You are marked on what you configure and on the evidence you produce that it works — not on a written description of what you would have done.

Declaration

I declare that the configuration and the evidence submitted with this paper are entirely my own work, produced by me in this session, and that I have not copied, shared or received terminal output from any other candidate.

Signature	<i>Syed</i>
Date	11/9/2026

1. General Instructions and Evidence Rules

1.1 Before you begin

- Confirm the cluster and context assigned to you, and include that confirmation in your first screenshot:

```
kubectl config current-context
kubectl cluster-info
```

- Start LocalStack and wait until it is actually serving. The container ID is returned long before port 4566 answers, and starting work too early produces a "Could not connect to the endpoint URL" error that is not your fault and wastes your time.

```
# Community edition -- no auth token, no licence, no cost
docker run -d --name localstack -p 4566:4566 localstack/localstack

# WAIT until it is healthy before running anything else
until curl -sf http://localhost:4566/_localstack/health >/dev/null; do sleep 2; done
echo "LocalStack ready"

# Dummy credentials -- LocalStack accepts any value
aws configure set aws_access_key_id test
aws configure set aws_secret_access_key test
aws configure set region us-east-1

# Set the endpoint once and reuse it in every command
EP='--endpoint-url=http://localhost:4566'

# Your first screenshot: which identity are you operating as?
aws $EP sts get-caller-identity
```

- Open your working terminal with your identity line. This line must be visible in the same frame as every screenshot you submit:

```
echo "<YOUR STUDENT ID> $(date)"
```

- If any command in this paper fails for a reason you believe is caused by the environment rather than by your own configuration — the container will not start, an image will not pull, port 4566 is already in use — raise your hand and inform the invigilator immediately. Do not spend your time working around it.

1.2 What counts as evidence

- Every screenshot must show the command you executed and its corresponding output, in one uncropped frame, together with your identity line.
- A screenshot of a manifest or a configuration file, with no verification output, earns configuration marks only. It can never earn verification marks.
- Screenshots must be legible. If the marker cannot read the command or the output, the evidence cannot be credited.
- You may optionally record a text transcript alongside your screenshots using `script -q ~/answers/questionN.log`, ending it with `exit`. A transcript is accepted as supporting evidence but it does not replace the required screenshots.
- Label each screenshot with the evidence number given in the question — Screenshot 1, Evidence 2, and so on.

1.3 How you will be marked

- You are marked on the effective security outcome, not on which syntax you chose. Imperative commands such as `kubectl create`, and declarative manifests applied with `kubectl apply -f`, receive identical credit when the result is the same.
- Grant no more permission or access than the task requires. A configuration that passes the stated verification commands but confers broader access than the scenario asked for will lose marks.
- Output wording varies slightly between Kubernetes versions. You are marked on the decision your evidence shows — yes or no, HTTP 200 or a timeout, an accepted or a rejected creation — and not on cosmetic differences in formatting.
- Resource names are case-sensitive. Use exactly the names given in your question.

1.4 Rules of conduct

- `kubectl --help` and `kubectl explain` are permitted unless your lecturer has declared this a closed-reference sitting.
- Do not print or decode the value of any Kubernetes Secret anywhere in your evidence. Authorization is proven by the authorization decision, not by disclosing the protected data. Marks are deducted if a Secret value appears in your submission.
- Your evidence must be your own. Identical screenshots across candidates — matching hostnames, timestamps, ClusterIP addresses or identity lines — are referred under the programme academic integrity policy and are not marked.
- Do not delete or modify any namespace, workload or policy that the question tells you to leave alone.

Question 1 · Cross-Tenant Secret Isolation

Total: 15 marks · Suggested time: 20 minutes

Scenario

The Finance and HR applications share one Kubernetes cluster. Each application stores sensitive configuration as Kubernetes Secrets. The Finance application must never be able to retrieve HR secrets.

Environment requirements

The requirements are as in the table below.

Item	Required value
Namespaces	finance and hr
Secret in finance	Name finance-db, key DB_PASSWORD, value DB_PASSWORD_FINANCE
Secret in hr	Name hr-db, key DB_PASSWORD, value DB_PASSWORD_HR
ServiceAccount	finance-app, in the finance namespace
Required permission	get on Secrets, in the finance namespace only

You are required to perform the tasks below which configure the above requirement:

Task

1. Create the two namespaces and their respective Secrets, exactly as specified in the table above.
2. Create the finance-app ServiceAccount.
3. Create the minimum RBAC resources required so that finance-app can get Secrets in finance.
4. Do not grant finance-app any access to the hr namespace.
5. Use kubectl auth can-i to verify both the allowed and the denied access paths.
6. Demonstrate that finance-app cannot list or delete Secrets even inside its own namespace.

Expected behaviour you should not mistake for a fault — The permission you are asked to grant is get, and get is not the same verb as list. A plural, unnamed request such as `kubectl get secrets -n finance` will therefore be refused even when your configuration is completely correct. To read a single Secret you must name it: `kubectl get secret finance-db -n finance`. Understanding this distinction is part of what is being assessed.

Do not disclose values — Your evidence must not print or decode any Secret value. Marks are deducted if it does.

Required evidence

Screenshot 1 — `get` Secrets in finance = yes, and `get` Secrets in hr = no, for the same identity.

Screenshot 2 — `list` Secrets and `delete` Secrets in finance both = no, together with one impersonated read of the finance Secret by name that succeeds, and one of the hr Secret that is refused.

Technical analysis

Identify the Kubernetes resources you used to enforce the boundary, and explain how namespace scope prevents cross-tenant secret access. [3 marks]

Mark allocation

Component	What is assessed	Marks
Setup	Namespaces, Secrets and ServiceAccount created exactly as specified	4
RBAC	Least-privilege authorization resources configured in the finance namespace, with no access to hr	5
Verification	Own-tenant access allowed, cross-tenant access denied, and the least-privilege negatives shown	3
Analysis	Correct explanation of namespaced RBAC and secret isolation	3
	Total	15

Question 2 · Resource Isolation — Protecting Against the Noisy Neighbour

Total: 15 marks · Suggested time: 20 minutes

Scenario

A shared Kubernetes cluster hosts multiple student workloads. The student-app namespace must be prevented from consuming excessive shared compute resources.

Security requirements

The security requirements that are required for you to configure:

Requirement	Maximum allocation for the namespace
Pods	5
CPU requests, aggregate	1 CPU
Memory requests, aggregate	512 MiB

Task

1. Create the student-app namespace.
2. Implement the appropriate Kubernetes resource control to enforce all three requirements above.
3. Verify that the control is active, and display its configured limits together with current usage.
4. Demonstrate enforcement twice:
 - (a) create workloads until the Pod count ceiling is reached, then capture the rejection of the next Pod; and
 - (b) capture a rejection caused by exceeding the aggregate CPU or memory request ceiling.
5. Capture the Kubernetes output that demonstrates enforcement in each case.

Important — two different rejections — Because this control constrains CPU and memory requests, every Pod you create in this namespace must declare its own resource requests. A Pod submitted with no requests is refused with a message containing "must specify requests.cpu,requests.memory". That is a different rejection from exceeding the ceiling, and on its own it will not earn the enforcement marks. Your enforcement evidence must contain the phrase "exceeded quota".

A note on kubectl run — The --requests and --limits flags were removed from kubectl run in Kubernetes v1.24 and are no longer available. To create a Pod that declares resource requests, apply a Pod manifest, or use kubectl run with the --overrides flag.

Required evidence

Evidence 1 — The active resource control, showing the namespace, the Pod ceiling, the CPU request ceiling, the memory request ceiling, and current usage against each.

Evidence 2 — A workload creation rejected with an "exceeded quota" message, together with the state that makes it meaningful — the running Pods, or the current usage figures, immediately beforehand.

Technical analysis

Explain the noisy-neighbour problem, and how the Kubernetes control you configured reduces this risk in a shared multi-tenant environment. [3 marks]

Mark allocation

Component	What is assessed	Marks
Configuration	The correct resource control created in the correct namespace, with all three values set as specified	5
Verification	The control is visible, correctly scoped, and shows both usage and limits	3
Enforcement	Evidence of a genuine over-quota rejection, with the preceding state shown	4
Analysis	Correct explanation of the noisy-neighbour problem and of how the control mitigates it	3
	Total	15

Question 3 · Object Storage — Public Exposure and Data Remanence

Total: 20 marks · Suggested time: 30 minutes

Scenario

You are the cloud security engineer for a Malaysian health-tech company. Patient records are held in an S3 bucket alongside internal and public material. A colleague has been asked to "make the notice page readable by everyone" and is about to write a bucket policy. You will first reproduce the mistake they are about to make, then remediate it, and finally establish whether the data can actually be destroyed on request.

Environment requirements

Item	Required value
Bucket	<YOUR STUDENT ID>-patient-records, all lower case
Objects	public/notice.txt, internal/roster.txt, confidential/record.txt
Object tag	Classification — set to Public, Internal and Confidential respectively
Confidential content	Patient: Ahmad, Diagnosis: confidential
Endpoint	http://localhost:4566 — LocalStack Community, via \$EP

Part A — Create, classify, and reproduce the archetypal breach [6 marks]

1. Create the bucket and the three objects with the exact keys above.
2. Tag each object with its Classification. Verify the tagging of the confidential object from the API — not from your own notes.
3. Write a bucket policy that grants s3:GetObject to "Principal": "*" on every object in the bucket, and apply it.
4. Read the confidential object back **with no credentials at all** — no AWS CLI, no signature, just a URL — and capture the HTTP status code and the content that is returned.

```
BUCKET=<your-student-id>-patient-records

# The attacker's view: no credentials, just a URL
curl -s -o leaked.txt -w 'HTTP %{http_code}\n' \
  http://localhost:4566/$BUCKET/confidential/record.txt
cat leaked.txt
```

Evidence 1

The anonymous request returning **HTTP 200** and the patient record printed in your terminal, together with the get-bucket-policy output that caused it.

Short analysis. State the single word in the policy JSON that caused the exposure, and name the AWS access-control mechanism that was misused. [2 marks]

Part B — Remediate and install a guardrail [7 marks]

1. Remove the offending bucket policy.
2. Apply Block Public Access to the bucket with all four flags set to true, and display the stored configuration from the API.
3. Attempt to re-apply the **identical** public policy from Part A.
4. Re-run the **identical** anonymous read from Part A.
5. Write and apply the policy your colleague should have had: read access to the internal/ prefix only, for your own account principal only, and nothing else.

Evidence 2

The get-public-access-block output showing all four flags true, and the least-privilege policy read back from the bucket with get-bucket-policy.

Evidence 3

The re-applied public policy attempt and the re-run anonymous read, whatever their outcome.

Read this before you conclude anything is broken

LocalStack Community stores the Block Public Access configuration faithfully but **does not enforce it**. Step 3 may therefore succeed and step 4 may still return HTTP 200 on your machine. That is a known limitation of the simulator, not a defect in your configuration, and you will not lose marks for it.

What you must do is capture the outcome you actually observed and then answer, in your report: **(a)** which one of the four Block Public Access flags would have rejected the policy in step 3 on real AWS, and **(b)** why a preventative guardrail of this kind is a stronger control than a detective one that merely reports the bucket as public after the fact. [3 marks]

Part C — Data remanence: what "delete" really means [7 marks]

The company receives a request from a patient to erase their record. Before you promise anything, establish what deletion actually does in object storage.

1. Enable versioning on the bucket and confirm it from the API.
2. Overwrite confidential/record.txt once, so that two versions exist. List the versions.
3. Delete confidential/record.txt with an ordinary delete-object call.
4. Show that an ordinary get-object for that key now fails, **and** that the data is nevertheless still retrievable.
5. Now remove one specific version permanently, and show the version list changing to prove it is gone.

Evidence 4

The failed ordinary get-object, immediately followed by a successful get-object --version-id ... that returns the supposedly deleted patient record.

Evidence 5

The list-object-versions output before and after the permanent version deletion, with the delete marker visible.

Do not disclose more than you must

Part C requires you to show that the record is recoverable. Showing the **object metadata and the version id** is sufficient proof. You do not need to print the record body a second time — you already did that in Evidence 1.

Analysis. Explain why a delete-object call on a versioned bucket does not delete the data, what a delete marker is, and what this means for a company that has told a patient their record has been erased. [2 marks of the 7]

Mark allocation

Part	What is assessed	Marks
A	Bucket and three correctly-keyed objects created; classification tags applied and verified from the API	2
A	A public bucket policy correctly written and applied	2
A	Anonymous HTTP 200 read of the confidential object captured	1
A	Short analysis: the offending word and the mechanism named	1
B	Offending policy removed; Block Public Access applied with all four flags and shown from the API	2

Part	What is assessed	Marks
B	Least-privilege replacement policy: correct prefix, correct principal, no wider access	2
B	Re-application attempt and re-run anonymous read both captured, whatever the outcome	1
B	Written answer: the correct flag identified, and prevention argued against detection	2
C	Versioning enabled and confirmed; two versions demonstrated	2
C	Ordinary get-object fails after deletion, and versioned get-object recovers the data	2
C	Delete marker identified in the version listing; a specific version permanently removed	1
C	Analysis of delete markers and the consequence for an erasure request	2
	TOTAL	20

Question 4 · Key Management — Envelope Encryption and Cryptographic Erasure

Total: 15 marks · Suggested time: 20 minutes

Scenario

The same company holds one encrypted archive per tenant. Legal has asked whether a tenant's archive can be made **provably** unrecoverable without physically destroying the disks it sits on — which, in a shared cloud, nobody can do. You will build the envelope-encryption scheme that makes this possible, prove it works, and then destroy the data by destroying the key.

Environment requirements

Item	Required value
Master key	A KMS customer master key described as <YOUR STUDENT ID> tenant-A master key
Plaintext file	record.txt, containing Patient: Ahmad, Diagnosis: confidential
Data key	AES-256, obtained from KMS with generate-data-key
Encrypted archive	record.env.enc
Wrapped data key	datakey.enc — raw bytes, not base64 text

Task — Build the envelope [5 marks]

1. Create the master key and capture its KeyId.
2. Ask KMS for an AES-256 data key. Save the plaintext copy and the wrapped copy to disk.
3. Encrypt record.txt locally with the plaintext data key, producing record.env.enc.
4. **Destroy every plaintext copy of the data key on disk.** Only the wrapped copy may remain.

```
EP='--endpoint-url=http://localhost:4566'
echo 'Patient: Ahmad, Diagnosis: confidential' > record.txt

KEY_A=$(aws $EP kms create-key \
  --description '<YOUR STUDENT ID> tenant-A master key' \
  --query KeyMetadata.KeyId --output text)
echo "KEY_A=$KEY_A"

# Both copies come back base64-encoded, tab-separated
aws $EP kms generate-data-key --key-id $KEY_A --key-spec AES_256 \
  --query '[Plaintext,CiphertextBlob]' --output text > dk.txt

cut -f1 dk.txt | base64 -d > datakey.bin      # plaintext key, raw bytes
cut -f2 dk.txt | base64 -d > datakey.enc     # WRAPPED key, raw bytes
rm -f dk.txt

# openssl needs a single-line passphrase, so derive one from the raw key
base64 < datakey.bin | tr -d '\n' > datakey.pass
openssl enc -aes-256-cbc -pbkdf2 -salt \
  -in record.txt -out record.env.enc -pass file:./datakey.pass

# Destroy the plaintext key material. Keep ONLY datakey.enc
rm -f datakey.bin datakey.pass
ls -l record.env.enc datakey.enc
```

Evidence 1

The KeyId, the encrypted archive and the wrapped data key on disk, and a directory listing proving no plaintext data key remains.

Two mechanics that catch people out

`--output text` returns base64 strings. `fileb://` sends a file as **raw bytes**. You must therefore `base64 -d` the wrapped key before writing it to `datakey.enc`, or the later `kms decrypt` will reject it.

`openssl -pass file:` reads only the **first line** of the file, so a raw 32-byte key containing a newline byte would be silently truncated. Deriving a single-line base64 passphrase, as above, avoids this.

Task — Prove the round trip [3 marks]

Before you destroy anything, prove the scheme actually works. Send the wrapped key back to KMS to be unwrapped, use it to decrypt the archive, and show the recovered file matches the original.

```
aws $EP kms decrypt --ciphertext-blob fileb://datakey.enc \
  --query Plaintext --output text | tr -d '\n' > datakey.pass

openssl enc -d -aes-256-cbc -pbkdf2 \
  -in record.env.enc -out record.rec.txt -pass file:./datakey.pass

diff record.txt record.rec.txt && echo 'MATCH: envelope round trip successful'
rm -f datakey.pass record.rec.txt
```

Evidence 2

The MATCH confirmation. Your evidence must **not** show the decoded plaintext key material itself.

Task — Cryptographic erasure [4 marks]

1. Disable the master key and schedule it for deletion with the minimum pending window.
2. Confirm the key state from the API.
3. Attempt the **identical** unwrap you performed a moment ago, on the same `datakey.enc`.

```
aws $EP kms disable-key --key-id $KEY_A
aws $EP kms schedule-key-deletion --key-id $KEY_A --pending-window-in-days 7

aws $EP kms describe-key --key-id $KEY_A \
  --query 'KeyMetadata.[KeyId,Enabled,KeyState,DeletionDate]' --output table

# The identical unwrap, now that the key is gone:
aws $EP kms decrypt --ciphertext-blob fileb://datakey.enc 2>&1 | head -3
```

Evidence 3

The key state showing Disabled or PendingDeletion, immediately followed by the failed unwrap attempt on the same wrapped key.

If your unwrap still succeeds

Some LocalStack builds do not check key state on decrypt. If yours returns the key anyway, capture that outcome honestly, capture the `describe-key` state as your Evidence 3, and state in your analysis what real AWS returns instead and why. You will not lose marks for the simulator's behaviour — you will lose marks for reporting an outcome you did not observe.

Technical analysis [3 marks]

1. Explain envelope encryption in your own words, and say why only the master key needs hardware-grade protection even though the archive may be terabytes. [1 mark]
2. Explain how cryptographic erasure achieves provable deletion in a shared cloud where overwriting the physical storage is impossible. [1 mark]

3. State one circumstance in which cryptographic erasure would **fail** to make the data unrecoverable. [1 mark]

Mark allocation

Component	What is assessed	Marks
Configuration	Master key created with the required description; AES-256 data key obtained from KMS	2
Configuration	Archive encrypted with the data key; wrapped key stored as raw bytes; all plaintext key material destroyed	3
Verification	The wrapped key unwrapped by KMS and the archive recovered, with the MATCH confirmation	3
Erasure	Key disabled and scheduled for deletion; key state shown from the API	2
Erasure	The identical unwrap attempted after erasure and its outcome captured honestly	2
Analysis	Three short technical answers	3
	TOTAL	15

End of question paper

Setup:

1. Initialise Environment

```
+ Practical_Test git:(main) x kubectl config current-context
kubectl cluster-info

kind-ccse-lab4
Kubernetes control plane is running at https://127.0.0.1:44125
CoreDNS is running at https://127.0.0.1:44125/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
+ Practical_Test git:(main) x until curl -sf http://localhost:4566/_localstack/health >/dev/null; do sleep 2; done
echo "LocalStack ready"

LocalStack ready
+ Practical_Test git:(main) x aws configure set aws_access_key_id test
aws configure set aws_secret_access_key test
aws configure set region us-east-1

+ Practical_Test git:(main) x EP='--endpoint-url=http://localhost:4566'
+ Practical_Test git:(main) x aws $EP sts get-caller-identity
+ Practical_Test git:(main) x echo "52215225160 $(date)"
52215225160 Fri Sep 11 06:03:49 PM +08 2026
```

2. Get Caller Identity

```
{
  "UserId": "000000000000",
  "Account": "000000000000",
  "Arn": "arn:aws:iam::000000000000:root"
}
```

Answer for Q1:

1. Evidence 1

```
+ Practical_Test git:(main) x echo "52215225160 $(date)"
52215225160 Fri Sep 11 06:07:57 PM +08 2026
+ Practical_Test git:(main) x kubectl auth can-i get secrets -n finance --as=system:serviceaccount:finance:finance-app
yes
+ Practical_Test git:(main) x kubectl auth can-i get secrets -n hr --as=system:serviceaccount:finance:finance-app
no
```

2. Evidence 2

```
52215225160 Fri Sep 11 06:09:10 PM +08 2026
+ Practical_Test git:(main) x kubectl auth can-i list secrets -n finance --as=system:serviceaccount:finance:finance-app
no
+ Practical_Test git:(main) x kubectl auth can-i delete secrets -n finance --as=system:serviceaccount:finance:finance-app
no
+ Practical_Test git:(main) x kubectl get secret finance-db -n finance --as=system:serviceaccount:finance:finance-app
NAME      TYPE      DATA   AGE
finance-db Opaque    1       3m
+ Practical_Test git:(main) x kubectl get secret hr-db -n hr --as=system:serviceaccount:finance:finance-app
Error from server (Forbidden): secrets "hr-db" is forbidden: User "system:serviceaccount:finance:finance-app" cannot get resource "secrets" in API group "" in the namespace "hr"
```

3. Technical Analysis

To enforce the boundary, Kubernetes RBAC was used through the creation of a Role and a RoleBinding. The Role is namespace-scoped by nature, which means that the permission to get secrets can only be applied in the same namespace where the Role was created. Through the creation of the Role in the finance namespace and tying it to the finance-app ServiceAccount through RoleBinding, the authorization becomes limited to this namespace boundary. Namespace scope ensures cross-tenant protection from accessing the secrets since Kubernetes RBAC

works with a default deny principle. In this case, the ServiceAccount does not have any permissions to access the HR secrets, which prevents lateral movement to other namespaces.

Answers for Q2:

1. Evidence 1

```
+ Practical_Test git:(main) x echo "52215225160 $(date)"
52215225160 Fri Sep 11 06:17:21 PM +08 2026
kubect describe quota student-quota -n student-app
Name:      student-quota
Namespace: student-app
Resource   Used  Hard
-----
pods       0    5
requests.cpu 0    1
requests.memory 0  512Mi
+ Practical_Test git:(main) x for i in {1..5}; do
  kubect run pod-$i --image=nginx -n student-app \
  --overrides='{ "spec": { "containers": [ { "name": "nginx", "image": "nginx", "resources": { "requests": { "cpu": "100m", "memory": "50Mi" } } } ] } }'
done
pod/pod-1 created
pod/pod-2 created
pod/pod-3 created
pod/pod-4 created
pod/pod-5 created
+ Practical_Test git:(main) x kubect get pods -n student-app
NAME    READY   STATUS    RESTARTS   AGE
pod-1   1/1     Running   0           16s
pod-2   1/1     Running   0           16s
pod-3   1/1     Running   0           16s
pod-4   1/1     Running   0           16s
pod-5   1/1     Running   0           15s
```

2. Evidence 2

```
s1ght@slght:~/UniKL/IKB42603---CLOUD-COMPUTING-SECURITY-ESSENTIALS/Practical_Test
Session Actions Edit View Help
+ Practical_Test git:(main) x echo "52215225160 $(date)"
52215225160 Fri Sep 11 06:19:31 PM +08 2026
+ Practical_Test git:(main) x kubect get pods -n student-app
NAME    READY   STATUS    RESTARTS   AGE
pod-1   1/1     Running   0           92s
pod-2   1/1     Running   0           92s
pod-3   1/1     Running   0           92s
pod-4   1/1     Running   0           92s
pod-5   1/1     Running   0           91s
+ Practical_Test git:(main) x kubect describe quota student-quota -n student-app
Name:      student-quota
Namespace: student-app
Resource   Used  Hard
-----
pods       5    5
requests.cpu 500m  1
requests.memory 250Mi 512Mi
+ Practical_Test git:(main) x kubect run pod-6 --image=nginx -n student-app \
  --overrides='{ "spec": { "containers": [ { "name": "nginx", "image": "nginx", "resources": { "requests": { "cpu": "100m", "memory": "50Mi" } } } ] } }'
Error from server (Forbidden): pods "pod-6" is forbidden: exceeded quota: student-quota, requested: pods=1, used: pods=5, limited: pods=5
+ Practical_Test git:(main) x kubect run memory-hog --image=nginx -n student-app \
  --overrides='{ "spec": { "containers": [ { "name": "nginx", "image": "nginx", "resources": { "requests": { "cpu": "100m", "memory": "600Mi" } } } ] } }'
Error from server (Forbidden): pods "memory-hog" is forbidden: exceeded quota: student-quota, requested: pods=1, requests.memory=600Mi, used: pods=5, requests.memory=250Mi, limited: pods=5, requests.memory=512Mi
```

3. Technical Analysis

In a multi-tenant environment of a cloud, the 'noisy neighbour' issue arises where a particular tenant's workloads tend to consume all the available CPU and memory resources, leaving nothing for others, thus impacting their performance and eventually resulting in a denial of service. In order to avoid such problems, I have created a Kubernetes ResourceQuota policy in the student-app namespace. Such a control ensures that there is a hard limit on the amount of resources that can be consumed collectively. This is achieved by limiting

the maximum number of Pods, CPU requests, and memory requests. The Kubernetes API server will deny any new request that crosses the set limits automatically.

Answer for Q3:

1. Evidence 1

```
+ Practical_Test git:(main) x echo "52215225160 $(date)"
52215225160 Fri Sep 11 06:29:26 PM +08 2026
+ Practical_Test git:(main) x curl -s -o leaked.txt -w 'HTTP %{http_code}\n' \
  http://localhost:4566/$BUCKET/confidential/record.txt
cat leaked.txt
echo "" # Just adding a newline for clean formatting
HTTP 200
Patient: Ahmad, Diagnosis: confidential

+ Practical_Test git:(main) x aws $EP s3api get-bucket-policy --bucket $BUCKET --query Policy --output text | jq .
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::52215225160-patient-records/*"
    }
  ]
}
```

2. Technical Analysis Part A

The one-word reason for the exposure of the system is the wildcard * that is added to the Principal element, allowing everyone, including non-authenticated individuals, to gain access. The AWS access control feature through which such an exposure was created is called the Bucket Policy.

3. Evidence 2

```
+ Practical_Test git:(main) x echo "52215225160 $(date)"
52215225160 Fri Sep 11 06:37:08 PM +08 2026
+ Practical_Test git:(main) x aws $EP s3api get-public-access-block --bucket $BUCKET
+ Practical_Test git:(main) x echo "{
  \"PublicAccessBlockConfiguration\": {
    \"BlockPublicAcls\": true,
    \"IgnorePublicAcls\": true,
    \"BlockPublicPolicy\": true,
    \"RestrictPublicBuckets\": true
  }
}"
{
  PublicAccessBlockConfiguration: {
    BlockPublicAcls: true,
    IgnorePublicAcls: true,
    BlockPublicPolicy: true,
    RestrictPublicBuckets: true
  }
}

+ Practical_Test git:(main) x aws $EP s3api get-bucket-policy --bucket $BUCKET --query Policy --output text | jq .
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::000000000000:root"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::52215225160-patient-records/internal/*"
    }
  ]
}
```


4. Evidence 3

```
+ Practical_Test git:(main) x echo "52215225160 $(date)"
52215225160 Fri Sep 11 06:39:43 PM +08 2026
+ Practical_Test git:(main) x aws $EP s3api put-bucket-policy --bucket $BUCKET --policy file://public-policy.json
+ Practical_Test git:(main) x curl -s -o leaked.txt -w 'HTTP %{http_code}\n' \
  http://localhost:4566/$BUCKET/confidential/record.txt
HTTP 200
```

5. Technical Analysis Part B

- i. In actual AWS infrastructure, the particular Block Public Access option that would have prevented the re-application of the public policy in step 3 would be BlockPublicPolicy.
- ii. The use of a guardrail like Block Public Access is considerably more powerful than the detective control since the former prevents the formation of the insecure configuration at the API level. The detective control only serves to notify the administrators that the bucket has been set to public access. In the cloud setting where there are automatic bots always scanning for any exposed storage, by the time an alert is raised, it means the data has been exposed.

6. Evidence

4

```
s1ght@s1ght:~/UniKL/IKB42603---CLOUD-COMPUTING-SECURITY-ESSENTIALS/Practical_Test
Session Actions Edit View Help
+ Practical_Test git:(main) x echo "52215225160 $(date)"
52215225160 Fri Sep 11 06:53:01 PM +08 2026
+ Practical_Test git:(main) x aws $EP s3api delete-object --bucket $BUCKET --key confidential/record.txt
+ Practical_Test git:(main) x echo "{
  \"DeleteMarker\": true,
  \"VersionId\": \"AaCQFQ5qDHdzes1S_5Kq3NlTVk30mQD2\"
}"
{
  DeleteMarker: true,
  VersionId: AaCQFQ5qDHdzes1S_5Kq3NlTVk30mQD2
}
+ Practical_Test git:(main) x aws $EP s3api get-object --bucket $BUCKET --key confidential/record.txt failed.txt
aws: [ERROR]: An error occurred (NoSuchKey) when calling the GetObject operation: The specified key does not exist.
+ Practical_Test git:(main) x VERSION_ID=$(aws $EP s3api list-object-versions --bucket $BUCKET --prefix confidential/record.txt --query 'Versions[-1].VersionId' --output text)
+ Practical_Test git:(main) x aws $EP s3api get-object --bucket $BUCKET --key confidential/record.txt --version-id $VERSION_ID recovered.txt
+ Practical_Test git:(main) x echo "{
  \"AcceptRanges\": \"bytes\",
  \"LastModified\": \"2026-09-11T10:26:14+00:00\",
  \"ContentLength\": 40,
  \"ETag\": \"\\\"0840fa3c43545045477729c1b77dc8a7\\\"\",
  \"ChecksumCRC64NVME\": \"Y5xHTuJOW3c=\",
  \"ChecksumType\": \"FULL_OBJECT\",
  \"VersionId\": \"null\",
  \"ContentType\": \"binary/octet-stream\",
  \"ServerSideEncryption\": \"AES256\",
  \"Metadata\": {},
  \"TagCount\": 1
}"
{
  AcceptRanges: bytes,
  LastModified: 2026-09-11T10:26:14+00:00,
  ContentLength: 40,
  ETag: "0840fa3c43545045477729c1b77dc8a7",
  ChecksumCRC64NVME: Y5xHTuJOW3c=,
  ChecksumType: FULL_OBJECT,
  VersionId: null,
  ContentType: binary/octet-stream,
  ServerSideEncryption: AES256,
  Metadata: {},
  TagCount: 1
}
```

7. Evidence 5

```

→ Practical_Test git:(main) x echo "52215225160 $(date)"
52215225160 Fri Sep 11 07:15:20 PM +08 2026
→ Practical_Test git:(main) x aws $EP s3api list-object-versions --bucket $BUCKET --prefix confidential/record.txt

→ Practical_Test git:(main) x echo "{
  \"Versions\": [
    {
      \"ETag\": \"\\\"0840fa3c43545045477729c1b77dc8a7\\\"\",
      \"ChecksumAlgorithm\": [
        \"CRC64NVME\"
      ],
      \"ChecksumType\": \"FULL_OBJECT\",
      \"Size\": 40,
      \"StorageClass\": \"STANDARD\",
      \"Key\": \"confidential/record.txt\",
      \"VersionId\": \"AaCQFQ5rYcTv6.KHkuYzW2Sejb2NTLxE\",
      \"IsLatest\": false,
      \"LastModified\": \"2026-09-11T11:15:07+00:00\",
      \"Owner\": {
        \"ID\": \"75aa57f09aa0c8caeab4f8c24e99d10f8e7faeebf76c078efc7c6caea54ba06a\"
      }
    }
  ],
  \"DeleteMarkers\": [
    {
      \"Owner\": {
        \"ID\": \"75aa57f09aa0c8caeab4f8c24e99d10f8e7faeebf76c078efc7c6caea54ba06a\"
      },
      \"Key\": \"confidential/record.txt\",
      \"VersionId\": \"AaCQFQ5s2V448LEp0vCY_mFxUo3GqRf0\",
      \"IsLatest\": true,
      \"LastModified\": \"2026-09-11T11:15:09+00:00\"
    },
    {
      \"Owner\": {
        \"ID\": \"75aa57f09aa0c8caeab4f8c24e99d10f8e7faeebf76c078efc7c6caea54ba06a\"
      },
      \"Key\": \"confidential/record.txt\",
      \"VersionId\": \"AaCQFQ5qDHdzes1S_5Kq3NlTVk30mQD2\",
      \"IsLatest\": false,
      \"LastModified\": \"2026-09-11T10:53:06+00:00\"
    }
  ],
  \"RequestCharged\": null,
  \"Prefix\": \"confidential/record.txt\"
}\"

→ Practical_Test git:(main) x aws $EP s3api delete-object --bucket $BUCKET --key confidential/record.txt --version
-id AaCQFQ5rYcTv6.KHkuYzW2Sejb2NTLxE
→ Practical_Test git:(main) x echo "{
  \"VersionId\": \"AaCQFQ5rYcTv6.KHkuYzW2Sejb2NTLxE\"
}\"
{
  VersionId: AaCQFQ5rYcTv6.KHkuYzW2Sejb2NTLxE
}

→ Practical_Test git:(main) x aws $EP s3api list-object-versions --bucket $BUCKET --prefix confidential/record.txt

→ Practical_Test git:(main) x echo "{
  \"DeleteMarkers\": [
    {
      \"Owner\": {
        \"ID\": \"75aa57f09aa0c8caeab4f8c24e99d10f8e7faeebf76c078efc7c6caea54ba06a\"
      },
      \"Key\": \"confidential/record.txt\",
      \"VersionId\": \"AaCQFQ5s2V448LEp0vCY_mFxUo3GqRf0\",
      \"IsLatest\": true,
      \"LastModified\": \"2026-09-11T11:15:09+00:00\"
    },
    {
      \"Owner\": {
        \"ID\": \"75aa57f09aa0c8caeab4f8c24e99d10f8e7faeebf76c078efc7c6caea54ba06a\"
      },
      \"Key\": \"confidential/record.txt\",
      \"VersionId\": \"AaCQFQ5qDHdzes1S_5Kq3NlTVk30mQD2\",
      \"IsLatest\": false,
      \"LastModified\": \"2026-09-11T10:53:06+00:00\"
    }
  ],
  \"RequestCharged\": null,
  \"Prefix\": \"confidential/record.txt\"
}\"

```

8. Technical Analysis Part C

With a versioned S3 bucket, a normal delete object will not erase the data but will create a Delete Marker at the top of the object layer. The Delete Marker will serve as a barrier for standard read operations, making them give an error message as “not found,” but the actual data stays intact and accessible through the versioning of the data. This translates to the idea that a normal delete operation does not erase the data on behalf of the patient's erasure request in a health-tech company. If the company claims that the data has been deleted when in reality all they have done is a normal delete, the company is in violation of data privacy compliance standards.

Answer for Q4:

1. Evidence 1

```
+ Practical_Test git:(main) x echo "52215225160 $(date)"
52215225160 Fri Sep 11 07:24:28 PM +08 2026
+ Practical_Test git:(main) x echo "KEY_A=$KEY_A"
KEY_A=a605cc19-c403-4f7e-bb28-3a9eb33ebf35
+ Practical_Test git:(main) x ls -l record.env.enc datakey.enc
ls -l datakey.bin datakey.pass 2>/dev/null || echo "Plaintext keys successfully destroyed"
-rw-rw-r-- 1 s1ght s1ght 116 Sep 11 19:23 datakey.enc
-rw-rw-r-- 1 s1ght s1ght 64 Sep 11 19:23 record.env.enc
Plaintext keys successfully destroyed
```

2. Evidence 2

```
s1ght@s1ght:~/UniKL/IKB42603---CLOUD-COMPUTING-SECURITY-ESSENTIALS/Practical_Test
Session Actions Edit View Help
+ Practical_Test git:(main) x echo "52215225160 $(date)"
52215225160 Fri Sep 11 07:27:16 PM +08 2026
+ Practical_Test git:(main) x diff record.txt record.rec.txt && echo 'MATCH: envelope round trip successful'
MATCH: envelope round trip successful
+ Practical_Test git:(main) x rm -f datakey.pass record.rec.txt
```

3. Evidence 3

```
s1ght@s1ght:~/UniKL/IKB42603---CLOUD-COMPUTING-SECURITY-ESSENTIALS/Practical_Test
Session Actions Edit View Help
+ Practical_Test git:(main) x echo "52215225160 $(date)"
52215225160 Fri Sep 11 07:30:25 PM +08 2026
+ Practical_Test git:(main) x aws $EP kms disable-key --key-id $KEY_A
aws $EP kms schedule-key-deletion --key-id $KEY_A --pending-window-in-days 7
+ Practical_Test git:(main) x echo "{
  \"KeyId\": \"a605cc19-c403-4f7e-bb28-3a9eb33ebf35\",
  \"DeletionDate\": \"2026-09-18T19:30:33.184119+08:00\",
  \"KeyState\": \"PendingDeletion\",
  \"PendingWindowInDays\": 7
}"
{
  KeyId: a605cc19-c403-4f7e-bb28-3a9eb33ebf35,
  DeletionDate: 2026-09-18T19:30:33.184119+08:00,
  KeyState: PendingDeletion,
  PendingWindowInDays: 7
}
+ Practical_Test git:(main) x aws $EP kms describe-key --key-id $KEY_A \
--query 'KeyMetadata.[KeyId,Enabled,KeyState,DeletionDate]' --output table
+ Practical_Test git:(main) x echo "-----"
DescribeKey
+-----+
| a605cc19-c403-4f7e-bb28-3a9eb33ebf35 |
| False |
| PendingDeletion |
| 2026-09-18T19:30:33.184119+08:00 |
+-----+
+-----+
DescribeKey
+-----+
| a605cc19-c403-4f7e-bb28-3a9eb33ebf35 |
| False |
| PendingDeletion |
| 2026-09-18T19:30:33.184119+08:00 |
+-----+
+ Practical_Test git:(main) x aws $EP kms decrypt --ciphertext-blob fileb://datakey.enc 2>&1 | head -3
aws: [ERROR]: An error occurred (KMSInvalidStateException) when calling the Decrypt operation: arn:aws:kms:us-east-1:000000000000:key/a605cc19-c403-4f7e-bb28-3a9eb33ebf35 is pending deletion.
```

4. Technical Analysis

- Envelope Encryption

The process of envelope encryption involves encrypting the data using the Data Key and wrapping the encrypted data key using the Master Key. Only the Master Key needs hardware-grade protection since the hard work of encryption of terabytes of data is done by the Data Key locally. The HSM will only have to do processing of the small Master Key and can keep the wrapped Data Key safe in any place.

- Cryptographic Erasure

Since the destruction of physical disk is not possible in a shared cloud environment, the cryptographic erasure will mathematically prove the deletion of the data. By destroying the Master Key, the unwrapping of the Data Key will become impossible hence making the ciphertext irretrievable.

- Failure Circumstance

This form of erasure will fail to delete the data irreversibly if there is a leak or backup of the plaintext form of the Data Key before it is destroyed.